Transactional Outbox

eason: 4 · Enterprise · Category: Enterprise · Level: Intermediate · Build with Waqa

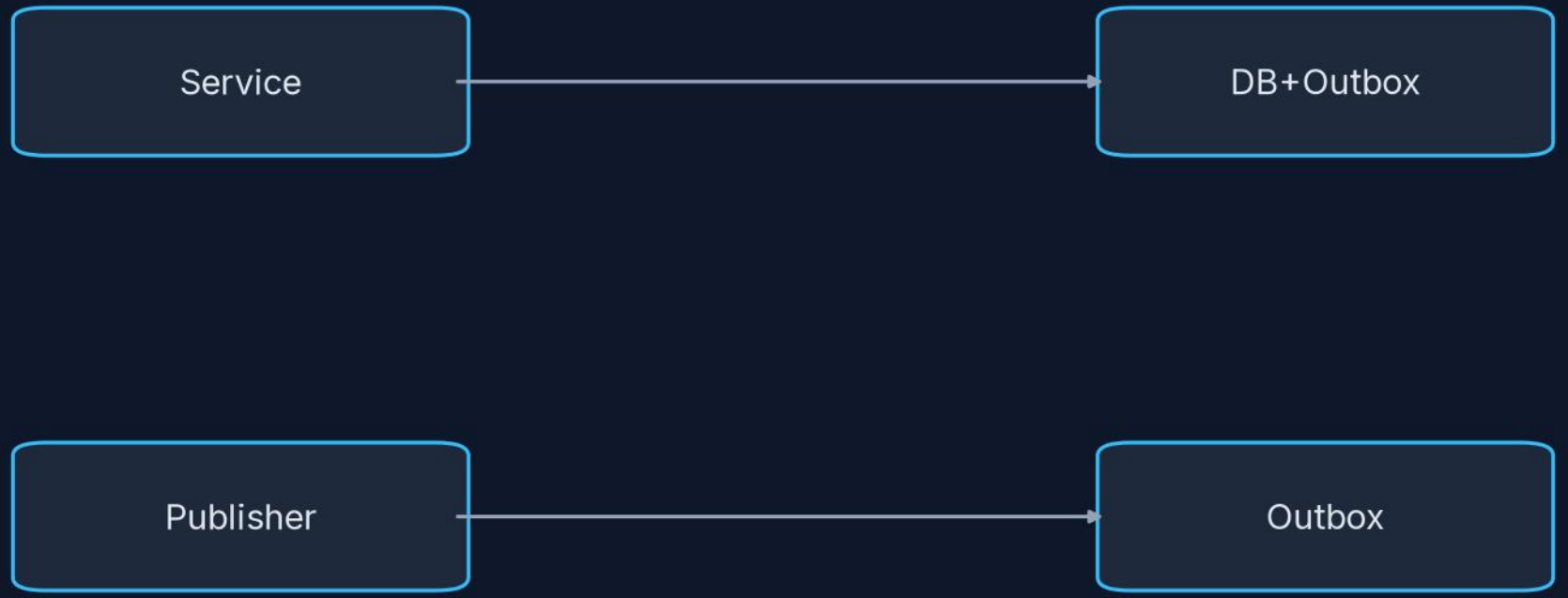
Today's Hook

- Today: Transactional Outbox as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Transactional Outbox structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Transactional Outbox
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/enterprise/outbox/Outbox.java
package com.buildwithwaqar.patterns.enterprise.outbox;

import java.util.ArrayList;
import java.util.List;

record OutboxMessage(String id, String topic, String payload, boolean published) {
    OutboxMessage markPublished() { return new OutboxMessage(id, topic, payload, true); }
}

/** Transactional outbox — persist business row + message atomically, publish async. */
public final class Outbox {
    private final List<String> orders = new ArrayList<>();
    private final List<OutboxMessage> messages = new ArrayList<>();

    public synchronized void placeOrder(String orderId, String payload) {
        orders.add(orderId);
        messages.add(new OutboxMessage(orderId, "orders", payload, false));
    }

    public synchronized List<OutboxMessage> unpublished() {
        return messages.stream().filter(m -> !m.published()).toList();
    }
```

Implementation (2)

```
    public synchronized void publishDue() {
        for (int i = 0; i < messages.size(); i++) {
            var m = messages.get(i);
            if (!m.published()) messages.set(i, m.markPublished());
        }
    }

    public List<String> orders() { return List.copyOf(orders); }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.enterprise.outbox;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Outbox walkthrough ===");
    var outbox = new Outbox();
    outbox.placeOrder("o1", "{\"sku\":\"course-1\"}");
    System.out.println("unpublished=" + outbox.unpublished().size());
    outbox.publishDue();
    System.out.println("unpublishedAfter=" + outbox.unpublished().size());
    System.out.println("orders=" + outbox.orders());
  }
}
```

Walkthrough Output

```
=== Outbox walkthrough ===
unpublished=1
unpublishedAfter=0
orders=[o1]
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Transactional Outbox
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Transactional Outbox → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.